Multi – Level linked list

–Insert At Position

– Time Complexity

Program:

```
void insertAtPosition()
{
    int data, pos;
    cout << "Enter data: ";
    cin >> data;
    cout << "Enter position (1-based): ";
    cin >> pos;

    Node *newNode = createNode(data);

    if (pos == 1)
    {
        newNode->next = head;
        head = newNode;
        return;
    }

    Node *temp = head;
    for (int i = 1; i < pos - 1 && temp != nullptr; i++)
    {
        temp = temp->next;
    }

    if (temp == nullptr)
    {
        cout << "Position out of range." << endl;
        free(newNode);
        return;
    }
```

```
    else
    {
        newNode->next = temp->next;
        temp->next = newNode;
        cout << "Inserted at position " << pos << endl;
    }
}
……
int main()
{
    int choice;
    while (true)
    {
      cout << "3. Insert at Position (Main)" << endl;
       cin >> choice;
        switch (choice)
        {
        case 3:
            insertAtPosition();
            break;
       default:
            cout << "Invalid choice" << endl;
        }
    }
    return 0;
}
```

Time Complexity

→ ***Absolute Best Case is when List is Empty*** : $\Omega(1)$
Though one may tell Best Case is when Node is inserted at first position , when List is not EMPTY But LINE OF CODE is less when List is Empty , hence absolute best case is when List is Empty.

→ ***Absolute Worst Case , when ``Position is Out of Bound***!``
The loop runs `n` times , hence : $O(n)$, [***1 time extra when Temp will be assigned to NULLPTR***.]

Though one may tell worst is when element is inserted at $N+1$ ***position , note the loop will be executed*** $n-1$ ***times while ``Position out of Bounds`` , loop runs `n` times , hence absolute worst case is*** :
``***Position Out Of Bounds***``.

Average Case

[***Based on Loop***]

3. ***Inserted position from*** 1 ***to*** $N+1$, ***while loop runs from*** 1 ***to*** $N-1$.

And when the position is uniformly distributed from 1 ***to*** $(N+1)$:

($Average\ Case\ i.e.\ Undetermined\ Position$)

→ ***If the position is*** 1, ***the loop's inner statement runs*** 0 ***time***.

→ ***If the position is*** 2, ***the loop's inner statement runs*** 0 ***time***.

→ ***If the position is*** 3, ***the loop's inner statement runs*** 1 ***times***.

→ ***If the position is*** 4, ***the loop's inner statement runs*** 2 ***times***.

.

.

→ ***If the position is*** $(N+1)$, ***the loop's inner statement runs*** : $N-1$ ***times*** , ***As when*** $i = N < N$ ***becomes false*** , ***hence no execution of loop's inner statements***.

$$Hence\ ,\ \sum_{i=1}^{N-1} i = \frac{N(N-1)}{2}$$

Now, as we take all the possibilites then , ***we take the best case also i.e.*** $O(1)$, ***when position =*** 1 ***and*** 2. ***Considering*** `$T(P)$` ***is time complexity of position*** `$P$` .

$$T(P) = \begin{cases} 1 & ,when\ P = 1. \\ 1 & ,when\ P = 2. \\ P-2\ or\ N-1 & ,when\ P = 3,4,..,N+1. \end{cases}$$

$$Now, If\ there\ is\ `N`\ no.\ of\ nodes\ there\ we\ can\ insert\ data\ at\ \ N+1\ positions.$$

$$For\ position\ 1, the\ probability\ of\ insertion\ is \frac{1}{N+1}.$$

$$For\ position\ 2, the\ probability\ of\ insertion\ \ is \frac{1}{N+1}.$$

.

.

$$For\ position\ N+1, the\ probability\ of\ insertion\ \ is \frac{1}{N+1}.$$

$$Hence\ probability\ of\ insertion\ will\ always\ remain\ \frac{1}{N+1}.$$

$$\therefore\ Average\ Time\ Complexity\ \ is:\ probability\ of\ insertion\ \times (number\ of\ possible\ iterations + constant\ time\ complexities\ for\ left\ over\ positions)\ .$$

$$= \left(\frac{1}{N+1}\times 1 + \frac{1}{N+1}\times 2 + \cdots + \frac{1}{N+1}\times N-1\right) + \frac{1}{N+1}\times 1 + \frac{1}{N+1}\times 1$$

$$= \frac{1}{N+1}\times\{(1+2+3+\cdots+N-1)+1+1\}$$

$$= \frac{1}{N+1}\times\left\{\sum_{i=1}^{N-1} i + 2\right\}$$

$$= \frac{1}{N+1}\times\left(\frac{N(N-1)}{2}+2\right)$$

$$= \frac{N(N-1)}{2(N+1)} + \frac{2}{N+1}$$

$$= \frac{N^2 - N}{2N + 2} + \frac{2}{N + 1}$$

Appplying Big Theta in the equation:

$$= \Theta\left(\frac{N^2 - N}{2N + 2}\right) + \Theta\left(\frac{2}{N + 1}\right)$$

$$= \Theta\left(\frac{N^2 - N}{2N + 2}\right) + \Theta\left(\frac{2}{N + 1}\right)$$

Now, taking out the dominant term of numerator and denominator:

- ***In the numerator,*** $N^2 - N$, ***the term*** N^2 ***grows much faster than*** N.

So, for large N, ***the dominant term is*** N^2.

- ***In the denominator,*** $2N + 2$, ***the term*** $2N$ ***grows much faster than the constant*** 2. ***The dominant term is*** $2N$.

$$= \Theta\left(\frac{N^2}{2N}\right) + \Theta\left(\frac{2}{N}\right)$$

$$= \Theta\left(\frac{N}{2}\right) + \Theta\left(\frac{2}{N}\right)$$

$$= \Theta\left(\frac{N}{2}\right) + 2 \times \Theta\left(\frac{1}{N}\right)$$

Now removing the constant `2` ***we get***:

$$= \Theta\left(\frac{N}{2}\right) + \Theta\left(\frac{1}{N}\right)$$

$$= \frac{1}{2} \times \Theta(N) + \Theta\left(\frac{1}{N}\right)$$

Now removing the constant `$\frac{1}{2}$` ***we get***:

$$= \Theta(N) + \Theta\left(\frac{1}{N}\right)$$

When combining two Big Theta Θ **terms , largest growth rate is the result**:

$\rightarrow \Theta(N)$ ***grows linearly with*** `N` .

$\rightarrow \Theta\left(\frac{1}{N}\right)$ ***decreases to a constant , i.e. when N grows larger ,*** $\frac{1}{N}$ ***approaches to*** 0.

This we can prove by limit and continuity also:

$\lim_{n\to\infty}\left(\frac{1}{n}\right) \Rightarrow \frac{1}{\infty} = 0$ $\left[\frac{c}{\infty} = 0 \text{ , where c is constant}\right]$.

As $\Theta(N)'s$ ***growth rate is higher , therefore , average case is*** $\Theta(N)$.

Hence, Average Time Complexity is : $\Theta(N)$.

$$Note\ , by\ relation: \Omega \leq \Theta \leq O\ , we\ get:$$

$$\Rightarrow c_1 \times g(n) \leq f(n) \leq c_2 \times g(n)$$

$$\Rightarrow c_1 \times 1 \leq \frac{N(N-1)}{2(N+1)} + \frac{2}{N+1} \leq c_2 \times N$$

$Average\ Case\ Time\ Complexity[Most\ Simplest\ Alternative\ Apporach]$:

$(Simplest\ approach)$

$$For\ position\ 1, the\ probability\ of\ insertion\ is \frac{1}{N+1}.$$

$$For\ position\ 2, the\ probability\ of\ insertion\ \ is \frac{1}{N+1}.$$

.

.

$$For\ position\ N+1, the\ probability\ of\ insertion\ \ is \frac{1}{N+1}.$$

$$Hence\ probability\ of\ insertion\ will\ always\ remain\ \frac{1}{N+1},$$

$$for\ position\ 1\ to\ N+1.$$

$$Average\ case\ complexity: \ Probability\ of\ insertion \times \begin{pmatrix} Summation \\ of\ number\ of \\ Position. \end{pmatrix}$$

$$= \frac{1}{N+1} \times \left(\sum_{i=1}^{N+1} i \right)$$

$$= \frac{1}{N+1} \times \left(\frac{1}{2}(N+1)\big((N+1)+1\big) \right) \left[As, \sum_{k=1}^{n} k = \frac{1}{2}(n)(n+1) \right]$$

$$= \frac{1}{N+1} \times \left(\frac{(N+1)(N+2)}{2} \right)$$

$$= \frac{N+2}{2}$$

$$= \frac{N}{2} + \frac{2}{2}$$

$$= \frac{N}{2} + 1$$

***Applying Big Theta* Θ we get:**

$$= \Theta\left(\frac{N}{2}\right) + \Theta(1)$$

$$= \frac{1}{2} \times \Theta(N)$$

$$= \Theta(N)$$

$$\textit{Hence, Average Time Complexity is : } \Theta\left(\frac{N}{2} + 1\right) \textit{ or } \Theta(N).$$

Now,

Note , by relation: $\Omega \leq \Theta \leq O$ ***, we get:***

$$\Longrightarrow c_1 \times g(n) \leq f(n) \leq c_2 \times g(n)$$

$$\Longrightarrow c_1 \times 1 \leq \frac{N}{2} + 1 \leq c_2 \times N$$

Multi − Level Linked List	***Cases***	***Time Complexity***
InsertAt Position	**1.** ***Best Case***	$\boldsymbol{\Omega(1)}$
	2. ***Worst Case***	$\boldsymbol{O(n)}$
	3. ***Average Case***	$\boldsymbol{\Theta(n)}$

****